

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия,
Системное и прикладное программное обеспечение

Дисциплина: Вычислительная математика

Лабораторная работа №1
Метод Гаусса-Зейделя
Вариант №6

Выполнил:
Карнажицкий Максим Романович
Группа: Р3211

Проверил:
Малышева Т. А.

г. Санкт-Петербург, 2025

Содержание

1. Цель работы	2
2. Описание метода, расчетные формулы	2
3. Листинг программы	3
3.1. Метод с решением системы	3
3.2. Приведение матрицы к диагональному преобладанию	5
3.3. Полный код	6
4. Результаты работы программы	6
5. Вывод	8

1. Цель работы

Используя известные методы вычислительной математики, написать программный код, осуществляющий решение СЛАУ, проанализировать полученные результаты и оценить погрешность.

2. Описание метода, расчетные формулы

Метод Гаусса-Зейделя является модификацией метода простых итераций и обеспечивает более быструю сходимость к решению систем линейных уравнений.

Рассмотрим СЛАУ с невырожденной матрицей ($\det A \neq 0$):

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \dots a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases} \quad (*)$$

Приведем (*) к виду (**), выразив неизвестные $x_1, \dots, x_n$ соответственно из первого, второго и т. д. уравнений системы (*):

$$\begin{cases} x_1 = -\frac{a_{12}}{a_{11}}x_2 - \frac{a_{13}}{a_{11}}x_3 - \dots - \frac{a_{1n}}{a_{11}}x_n + \frac{b_1}{a_{11}} \\ x_2 = -\frac{a_{21}}{a_{22}}x_1 - \frac{a_{23}}{a_{22}}x_3 - \dots - \frac{a_{2n}}{a_{22}}x_n + \frac{b_2}{a_{22}} \\ \dots \\ x_n = -\frac{a_{n1}}{a_{nn}}x_1 - \frac{a_{n2}}{a_{nn}}x_2 - \dots - \frac{a_{n,n-1}}{a_{nn}}x_{n-1} + \frac{b_n}{a_{nn}} \end{cases} \quad (**)$$

Обозначим

$$c_{ij} = \begin{cases} 0 & \text{при } i = j \\ -\frac{a_{ij}}{a_{ii}} & \text{при } i \neq j \end{cases}, d_i = \frac{b_i}{a_{ii}}, i = 1..n$$

Тогда получим:

$$\begin{cases} x_1 = c_{11}x_1 + c_{12}x_2 + \dots + c_{1n}x_n + d_1 \\ \dots \\ x_n = c_{n1}x_1 + c_{n2}x_2 + \dots + c_{nn}x_n + d_n \end{cases}$$

Что можно записать в векторно-матричном виде:

$$x = Cx + D$$

, C - матрица коэффициентов преобразованной системы (назовем ее **канонической формой**), D - вектор правых частей

Теперь можно записать формулу вычисления x_{ij} на k -ой итерации:

$$\begin{aligned}
x_i^{(k+1)} &= \sum_{j=1}^{i-1} (c_{ij}x_j^{(k+1)} + d_i) + \sum_{j=i}^n (c_{ij}x_j^{(k)} + d_i) \\
&= \frac{b_i}{a_{ii}} - \sum_{j=1}^{i-1} \left(\frac{a_{ij}}{a_{ii}} x_j^{(k+1)} \right) - \sum_{j=i}^n \left(\frac{a_{ij}}{a_{ii}} x_j^{(k)} \right)
\end{aligned}$$

Итерационный процесс продолжается до тех пор, пока:

$$\begin{cases} \left| x_1^{(k)} - x_1^{(k-1)} \right| \leq \varepsilon \\ \left| x_2^{(k)} - x_2^{(k-1)} \right| \leq \varepsilon \\ \dots \\ \left| x_n^{(k)} - x_n^{(k-1)} \right| \leq \varepsilon \end{cases} \Leftrightarrow \max \left(\left| x_1^{(k)} - x_1^{(k-1)} \right|, \dots, \left| x_n^{(k)} - x_n^{(k-1)} \right| \right) \leq \varepsilon$$

Полученные отклонения из последней системы называется **вектором погрешности**

3. Листинг программы

Логика решения систем написана на ЯП Go, и построена как клиент-серверное приложение с фронтендом на Vue.js 3 и backend на Golang с использованием библиотеки Chi для API и middleware.

3.1. Метод с решением системы

```

func (s *SeidelSolver) Solve(
    A matrix.Matrix,
    b matrix.Vector,
    x0 matrix.Vector,
    eps float64,
    maxIter int,
) (Result, error) {
    n := len(A)

    if len(b) != n {
        return Result{}, fmt.Errorf("неподходящие измерения: %d для A != %d для B", len(A), len(b))
    }
    if len(x0) != n {
        return Result{}, fmt.Errorf("начальное приближение имеет размерность %d != %d у матрицы A", len(x0), len(A))
    }

    ACopy := matrix.CopyMatrix(A)
    bCopy := matrix.CopyVector(b)

    if !matrix.TryToMakeDiagonallyDominant(ACopy, bCopy) {

```

```

    return Result{}, fmt.Errorf("матрицу A не удалось привести к
диагональному преобладанию")
}

C, d, err := matrix.BuildCanonicalForm(ACopy, bCopy)
if err != nil {
    return Result{}, err
}

normC := matrix.NormOfMatrix(C)
if normC >= 1 {
    return Result{}, fmt.Errorf("||C|| = %.6f >= 1, сходимость не
гарантирована", normC)
}

x := matrix.CopyVector(x0) // текущее решение
xPrev := matrix.NewVector(n) // прошлое решение
errors := []float64{} // вектор погрешностей

for iter := 0; iter < maxIter; iter++ {
    copy(xPrev, x)

    for i := 0; i < n; i++ {
        sum := d[i]

        // подстановка новых приближений (j < i)
        for j := 0; j < i; j++ {
            sum += C[i][j] * x[j]
        }

        // подстановка старых приближений (j > i)
        for j := i + 1; j < n; j++ {
            sum += C[i][j] * x[j]
        }

        x[i] = sum
    }

    diff, _ := matrix.SubVectors(x, xPrev)
    normVal := matrix.MaxNormOfVector(diff)
    errors = append(errors, normVal)

    if normVal < eps {
        return Result{
            Solution:    x,
            Iterations:  iter + 1,
        }
    }
}

```

```

        Errors:      errors,
        NormOfMatrix: normC,
    }, nil
}
}

return Result{}, fmt.Errorf("не сошлось за %d итераций. Последняя
погрешность: %.6f", maxIter, errors[len(errors)-1])
}

```

3.2. Приведение матрицы к диагональному преобладанию

```

// проверка диагонального преобладания
func IsDiagonallyDominant(A Matrix) bool {
    for i := range A {
        diag := math.Abs(A[i][i])
        sumExceptDiag := 0.0
        for j, val := range A[i] {
            if i != j {
                sumExceptDiag += math.Abs(val)
            }
        }
        if diag <= sumExceptDiag {
            return false
        }
    }
    return true
}

// применяем перестановку
func applyPermutation(A Matrix, b Vector, index []int) {
    n := len(A)

    ACopy := CopyMatrix(A)
    bCopy := CopyVector(b)

    for i := 0; i < n; i++ {
        copy(A[i], ACopy[index[i]])
        b[i] = bCopy[index[i]]
    }
}

// модификация IsDiagonallyDominant на перестановки
func checkPermutation(A Matrix, b Vector, index []int) bool {
    ACopy := CopyMatrix(A)
    bCopy := CopyVector(b)

```

```

    applyPermutation(ACopy, bCopy, index)
    return IsDiagonallyDominant(ACopy)
}

// рекурсивная проверка всех перестановок
func tryPermutations(A Matrix, b Vector, index []int, start int) bool {
    n := len(A)

    if start == n {
        if checkPermutation(A, b, index) {
            applyPermutation(A, b, index)
            return true
        }
        return false
    }

    for i := start; i < n; i++ {
        index[start], index[i] = index[i], index[start]
        if tryPermutations(A, b, index, start+1) {
            return true
        }
        index[start], index[i] = index[i], index[start]
    }

    return false
}

```

Получается, что асимптотика $O(n!)$, потому что мы перебираем все возможные перестановки рекурсивно до тех пор, пока не найдем подходящую.

3.3. Полный код

Полный код решения см. на Github

4. Результаты работы программы

Если пользователь выбирает вариант заполнения из файла, то формат файла следующий:

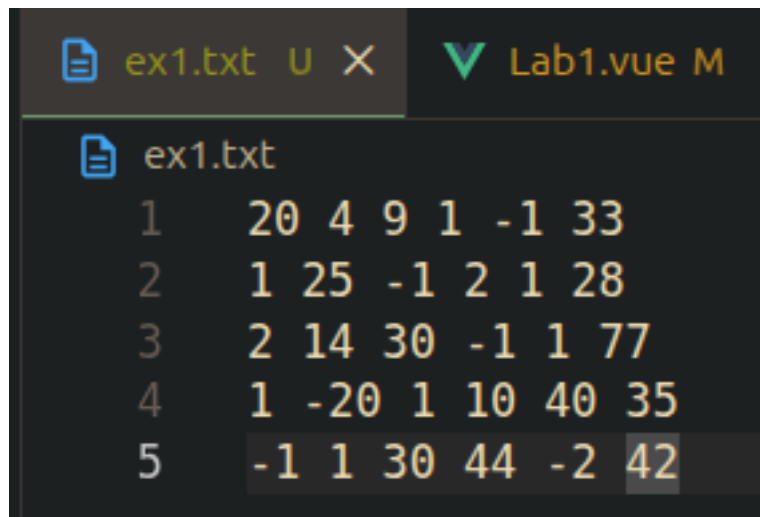


Рис. 1. Пример входных данных из файла

Первые n столбцов - матрица A с коэффициентами, последний столбец - вектор b .

Для запуска программы запускаем backend:

```
cd vm_lab1/backend
go run cmd/server/main.go
```

А затем frontend:

```
cd vm_frontend/vm_frontend
npm install
npm run dev
```

Пример работы программы:

Точность (eps):
Макс. итераций:

Введите систему $Ax = b$:

Матрица A	Вектор b	Начальное приближение
<pre> 20 4 9 1 -1 1 25 -1 2 1 2 14 30 -1 1 1 -20 1 10 40 -1 1 30 44 -2 </pre>	<pre> 33 28 77 35 42 </pre>	Опционально...

Вывод:

```

> найденное решение:
x_1 = 0.640389992506541
x_2 = 1.138194521869631
x_3 = 1.9339438035812664
x_4 = -0.3091379808740827
x_5 = 1.4570234112511409
> количество итераций: 5
> погрешности на каждой итерации:
1: 1.9648000000000003
2: 1.0061569090909095
3: 0.008583837548156614
4: 0.00046684757469039884
5: 0.000046334395682690044
> норма приведенной матрицы C: 0.8

```

Рис. 2. Пример корректной работы программы

5. Вывод

В ходе лабораторной работы я реализовал метод Гаусса-Зейделя, являющийся модификацией метода простых итераций, позволяющий решить СЛАУ с высокой (заданной) точностью. Сам метод не сложен в программном исполнении, самое сложное было реализовать приведение матрицы к диагональному преобладанию.